

Department of Computer Science

University of Engineering and Technology Lahore Pakistan

هم نوا

Hamnawa

Accessible Document Reader for Visually Impaired Students

Object-Oriented Design Model

Subject: Software Engineering **Group No.:** 2

Submitted by:

Muhammad Sarim	2025-CS-191
Abdullah Akbar	2025-CS-212
Muhammad Junaid Amir	2025-CS-193
Muhammad Ahmad Chaudhry	2025-CS-184

Submitted to:

Dr. Maimoona Salam

October 31, 2026

Document Control

Project	Hamnawa (ہم نوا) — Accessible Document Reader for Visually Impaired Students
Document	Object-Oriented Design Model, version 0.1 (draft)
Course	Software Engineering, Department of Computer Science, UET Lahore
Group	Group 2
Supervisor	Dr. Maimoona Salam
Based on	Requirements v0.1; System Architecture Document v0.1; Design Architecture Document v0.1
Status	Draft for supervisor review

Revision History

Version	Date	Description	Author(s)
0.1	October 31, 2026	First draft of the object-oriented design model.	Group 2
0.2	[date]	[After supervisor feedback]	[names]

The design model in one line. Hamnawa is designed with the **object-oriented design model**, expressed in UML: the system is a set of small, cohesive classes with clear responsibilities, programmed against interfaces, connected by a few well-known design patterns (Composite, Strategy, Adapter, Observer, Command, State and Facade). We chose it because the problem is naturally made of objects (documents, sections, bookmarks, a speech engine), and because this model gives the testability, replaceability and maintainability our requirements demand.

Contents

1 Introduction **4**

1.1 Purpose 4

1.2 Scope 4

1.3 How to Read This Document 4

2 Summary of the Design Model **4**

3 Why an Object-Oriented Design Model? **5**

3.1 Reasons 5

3.2 Alternatives Considered 5

4 Object-Oriented Concepts Applied **6**

5 Design Principles Followed **7**

6 Static Model (Class View) **8**

6.1 Types of Design Classes 8

6.2 UML Notation Used 9

6.3 Domain Model: the Document 9

6.4 Interfaces at the Boundaries 10

6.5 Class Specifications 11

6.6 Quality of the Classes 12

7 Design Patterns Used **13**

7.1 Observer: Event Flow 13

8 Dynamic Model (Behaviour) **14**

8.1 State of the Reader 14

8.2 Interaction: Open a Document and Read It 15

9 From Model to Code and Tests **15**

9.1 Mapping to Source Files 15

9.2 Testing Implications 16

10 Trade-offs and Limitations **16**

11 Traceability **17**

A Glossary **17**

1 Introduction

1.1 Purpose

This document describes the *object-oriented (OO) design model* of Hamnawa and explains *why* we used it. It turns the requirements and the system architecture into classes, interfaces, relationships and behaviour that a programmer can implement directly.

1.2 Scope

The Design Architecture Document decided *how the system is packaged* (a client-side modular monolith). The System Architecture Document decided *which components exist* (layers, CRC cards, data model, use cases). This document goes one level lower: it fixes the *classes and their collaboration inside the components*. Class names are the same as in the CRC cards, so the documents can be read together.

1.3 How to Read This Document

Section 2 gives the answer. Sections 3 to 5 justify it. Sections 6 to 8 present the model itself (static and dynamic views). Sections 9 to 11 connect it to code, tests and requirements.

2 Summary of the Design Model

Following Pressman and Maxim, the software design model has four elements. The table shows where each one lives for Hamnawa and which approach we used.

Design element	What we produced	Where
Data design	Document model (document, sections, content blocks); stored records (bookmark, reading state, settings); ERD	Section 6.3; System Architecture Document
Architectural design	Layers, document-processing pipeline, modular monolith	System Architecture Document; Design Architecture Document
Interface design	Three regions, keyboard shortcuts, voice commands, spoken announcements	Requirements document (FR-26 to FR-47); `UIController`, `Announcer`
Component-level design	**Classes, interfaces, patterns, state and sequence behaviour (this document)**	Sections 6–8

Design model chosen. Object-oriented design, modelled in UML (class, state and sequence diagrams), guided by encapsulation, abstraction, polymorphism, composition over inheritance and the SOLID principles. Design patterns are used only where they solve a real problem in Hamnawa.

3 Why an Object-Oriented Design Model?

3.1 Reasons

1. **The problem is made of objects.** The vocabulary of the requirements is already object-like: a *document* has *sections*; a section has *content blocks*; a user has *bookmarks*, a *reading position* and *settings*; the system uses a *speech engine*. Classes can mirror these nouns directly, so the design is easy to explain to the supervisor and to the team (Pressman's "design classes" come from the analysis classes).
2. **Change is expected, and objects hide change.** PDF and PPTX today; other formats later. Browser speech today; another engine tomorrow. When each such decision lives behind an interface, a change touches one class instead of many (NFR-M1, NFR-M2).
3. **State belongs with behaviour.** Reading has state (playing, paused, position, speed). Keeping state and the operations that change it in one class (`PlaybackController`) prevents the scattered flags and global variables that make speech applications unreliable.
4. **Testing without a browser.** Because speech, storage and file parsing are reached through interfaces, tests can substitute fakes. We can test "read the next sentence" without a microphone, a speaker or a real PDF (NFR-M3 and the test plan).
5. **One model for the whole team.** The CRC cards, ERD and use cases already use an object vocabulary. A UML class model continues the same thread, so the work of four people fits together.
6. **It matches the tools.** The chosen implementation language (TypeScript) has classes and interfaces. The design can be coded almost one-to-one, which keeps design and code from drifting apart.
7. **It is the standard taught in the course.** Pressman and Maxim present component-level design for object-oriented systems through classes, interfaces, cohesion, coupling and UML. Using it keeps our documents consistent with the course.

3.2 Alternatives Considered

Approach	Idea	Why we did not choose it as the main model
Structured (function-oriented) design	Break the system into functions and data flow diagrams.	Shared state (position, settings, document) would be passed around or made global. Adding a format or an engine would change many functions.
Pure functional design	Immutable data and pure functions.	A good fit for the text-cleaning steps, and we use it there. But reading, speech and focus are inherently stateful and event-driven, which objects model more naturally.
Component or scripting style (UI framework only)	Let the UI framework's components hold all logic.	Mixes interface and logic, which makes core behaviour hard to test without a browser.
Data-centred (database) design	Put the logic around a central data store.	Hamnawa has little persistent data and no server; the important complexity is behaviour, not storage.

Pragmatic note. The design is object-oriented, not object-obsessed. Stateless helpers, such as text-cleaning rules, are plain functions. Objects are used where there is state, a responsibility to protect, or a change to hide.

4 Object-Oriented Concepts Applied

Concept	Meaning	How Hamnawa uses it
Encapsulation	An object hides its data and exposes operations.	`PlaybackController` owns the current position; others cannot change it directly, they call `play()`, `next()` and similar operations. `BookmarkManager` is the only class that changes bookmarks.
Abstraction	Show what an object does, hide how.	`SpeechSynthesizer` says "speak this text"; callers do not know about browser details, voices or quirks.
Inheritance	A subclass specialises a parent.	`Heading`, `Paragraph`, `ListItem` and `Notice` specialise `ContentBlock`. Inheritance is used only for true "is-a" relations of the document model.

Concept	Meaning	How Hamnawa uses it
Polymorphism	One call, different behaviour by type.	The reader calls `cue()` on every block. A heading answers "Heading level 2", a list item answers "List item", without any `if` on the block type.
Composition	An object is built from parts that it owns.	A `DocumentModel` owns its sections; a section owns its blocks. Deleting the document discards all of them.
Interface (contract)	A promise of operations, without code.	`DocumentExtractor`, `SpeechSynthesizer`, `SpeechRecognizer` and `Storage`.
Association	Objects know and use each other.	Controllers use the document model and the interfaces above.

5 Design Principles Followed

Principle	Application in the design	Benefit
Single Responsibility	Each class has one reason to change: `BookmarkManager` manages bookmarks only; `Announcer` only delivers messages.	Small, understandable classes; easy review.
Open/Closed	Adding a format means adding one new extractor class; `DocumentLoader` is not edited.	Safe extension.
Liskov Substitution	Any `SpeechSynthesizer` (real or fake) can be used wherever one is expected, with the same promises.	Fakes are valid in tests.
Interface Segregation	Four small interfaces, not one large "Platform" interface.	Classes depend only on what they use.
Dependency Inversion	Controllers depend on interfaces; the concrete adapter is supplied when the application starts (dependency injection).	Replaceable engines; test doubles.
Composition over inheritance	Only one inheritance tree (content blocks). Everything else uses composition and interfaces.	Avoids fragile class hierarchies.

Principle	Application in the design	Benefit
High cohesion, low coupling	Related operations stay together; classes talk through interfaces and events.	Fewer ripple effects of change.
Information hiding	Browser APIs appear only inside the adapters.	Browser differences stay in one place.
Separation of concerns	Interface, application logic, domain and infrastructure are kept in four layers.	Matches the architecture.

6 Static Model (Class View)

6.1 Types of Design Classes

Pressman groups design classes into five types. Our classes fall into them as follows.

Class type	Purpose	Hamnawa classes
User interface	Everything the user sees, hears or presses.	`UIController`, `Announcer`
Business domain	The real things of the problem.	`DocumentModel`, `Section`, `ContentBlock` and its subclasses, `Bookmark`, `ReadingState`, `Settings`
Process	Carry out business rules and workflows.	`DocumentLoader`, `StructureAnalyzer`, `PlaybackController`, `NavigationController`, `VoiceCommandController`, `CommandParser`, `BookmarkManager`, `ResumeService`, `SettingsManager`
Persistent	Data that outlives a session.	`Bookmark`, `ReadingState`, `Settings` (stored through `Storage`)
System	Contact with the outside world.	`SpeechSynthesizerAdapter`, `SpeechRecognizerAdapter`, `StorageService`, `EventBus`, extractors

6.2 UML Notation Used



Figure 1: Notation used in the class diagrams.

6.3 Domain Model: the Document

The document model is the heart of the system. It is the same for PDF and PPTX, so everything after extraction (reading, navigation, bookmarks) is format-independent. A section can hold sub-sections, and content blocks specialise a common parent. This is the *Composite* structure: the reader walks the same tree whatever the file was.

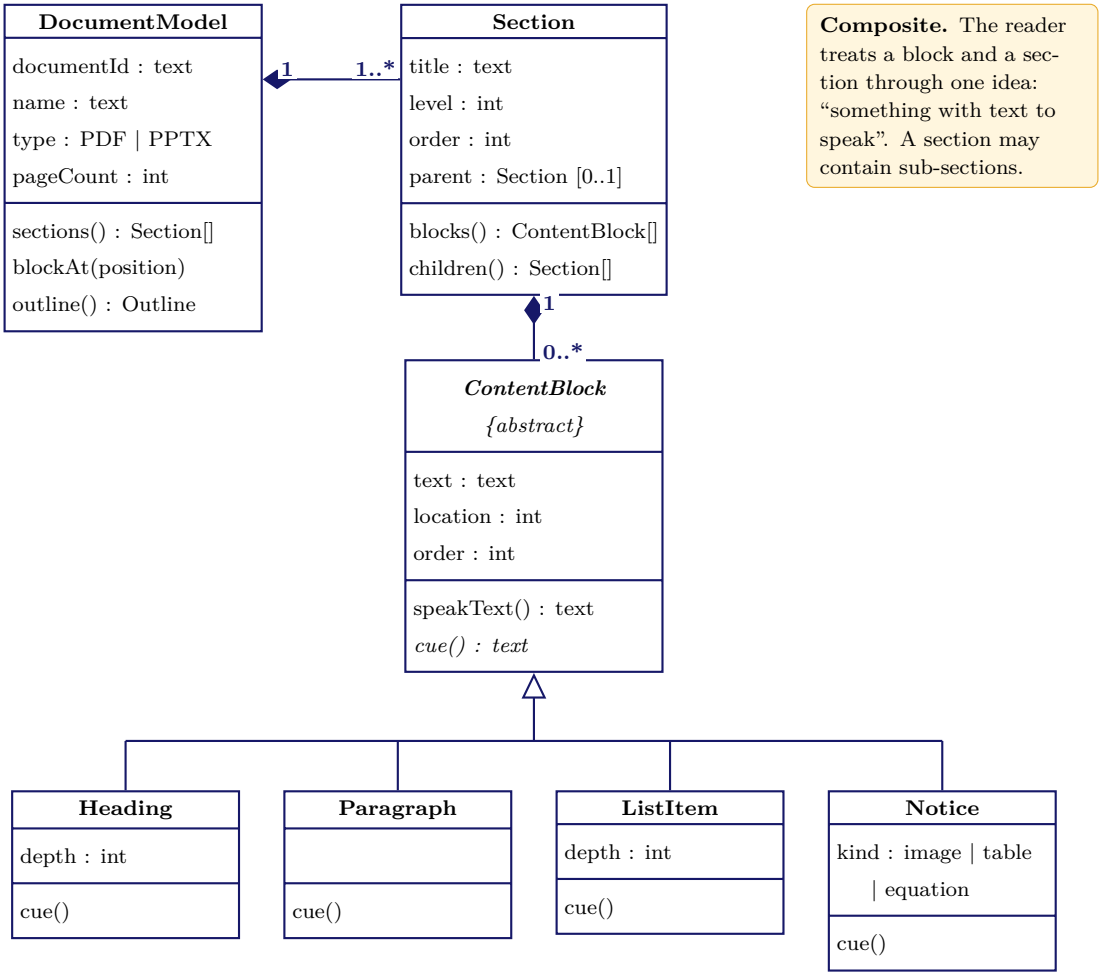


Figure 2: Class diagram of the document model. `cue()` is overridden in each subclass (polymorphism).

Why this shape.

- `DocumentModel` *owns* its sections, and a section *owns* its blocks (composition), so there are no dangling parts and memory is released when the document is closed.
- `ContentBlock` is abstract. The reader never asks "what kind of block is this?"; it calls `cue()` and gets the right spoken label for headings, lists and notices (FR-23).

- `Notice` stands for images, tables and equations that cannot be read. It lets the reader announce "image, no description" instead of silently skipping content (FR-08).
- `Bookmark`, `ReadingState` and `Settings` are simple records. They are stored through the `Storage` interface and are described in the ERD of the System Architecture Document.

6.4 Interfaces at the Boundaries

Four interfaces separate our logic from everything that can change or fail: the file format, the speech engine, the microphone and the storage. The classes on the left depend only on the interface; the classes on the right implement it. In tests, the *fake* on the right replaces the real adapter.

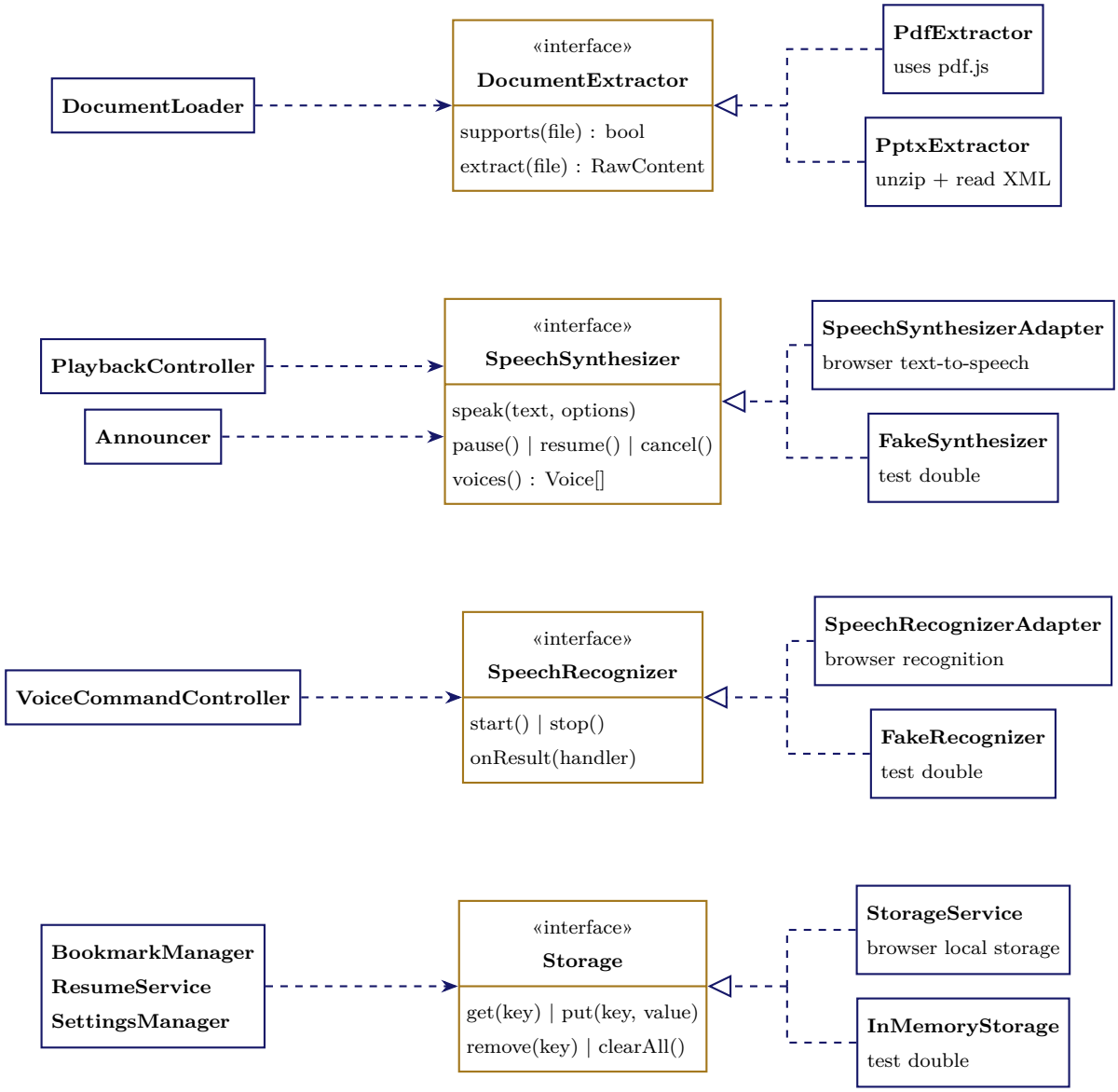


Figure 3: Interfaces at the boundaries. Dashed arrows from left show use; dashed triangles from right show implementation. Controllers never mention a browser feature.

Why this matters.

- **Open/closed.** Supporting a new format (for example EPUB) means adding one `EpubExtractor`. `DocumentLoader` chooses the extractor whose `supports(file)` is true (Strategy).
- **Failure containment.** A browser speech failure is caught in `SpeechSynthesizerAdapter` and reported through one error path; controllers never see browser-specific errors (NFR-R1).
- **Test doubles.** `FakeSynthesizer` records what would have been spoken, so a test can assert it without audio.

6.5 Class Specifications

The main process classes are specified below. Visibility: attributes are private; operations listed are the public interface. Full responsibilities and collaborators are on the CRC cards of the System Architecture Document.

Class	Attributes (private)	Public operations
DocumentLoader (Facade)	extractors : DocumentExtractor[]; analyzer; limits	`load(file) : DocumentModel`; `cancel()`; publishes *ProgressChanged*, *DocumentReady*, *LoadFailed*
StructureAnalyzer	rules : CleaningRule[]	`structure(raw) : DocumentModel`
PlaybackController	state : PlaybackState; position; model; synthesizer; settings	`play()`, `pause()`, `resume()`, `stop()`, `repeat()`, `next(unit)`, `previous(unit)`, `setRate(r)`
NavigationController	model; position	`nextSection()`, `previousSection()`, `goTo(n)`, `openOutline()`, `whereAmI() : text`, `search(term)`
VoiceCommandController	recognizer; parser; listening : bool	`enable()`, `disable()`; executes the `Command` returned by the parser
CommandParser	phrases : Map	`parse(phrase) : Command | Unknown`
BookmarkManager	bookmarks : Bookmark[]; storage	`add(label)`, `list()`, `remove(id)`, `jumpTo(id)`
ResumeService	storage	`savePosition(pos)`, `lastPosition(docId)`, `recentDocuments()`

Class	Attributes (private)	Public operations
SettingsManager	settings : Settings; storage	`get()`, `update(change)`, `eraseAll()`
Announcer	queue : Message[]; synthesizer; liveRegion	`say(text, priority)`, `error(text)`, `progress(percent)`
UIController	regions; focus order; shortcut map	`showReader()`, `showOutline()`, `highlight(sentence)`, `applyDisplay(settings)`, `showHelp()`
EventBus	subscribers : Map	`publish(event)`, `subscribe(type, handler)`, `unsubscribe(handler)`

6.6 Quality of the Classes

We checked the classes against the four characteristics of a well-formed design class given by Pressman and Maxim.

Characteristic	Check we applied	Example
Complete and sufficient	Each class has all operations needed for its responsibility, and no more.	`BookmarkManager` offers add, list, remove, jump.
Primitiveness	An operation does one thing; complex behaviour is built by combining operations.	`next(unit)` moves; `play()` speaks; neither does both.
High cohesion	All attributes and operations serve one purpose.	`ResumeService` only remembers positions.
Low coupling	Classes know interfaces and events, not each other's insides.	`PlaybackController` announces position changes as events; it does not call `UIController`.

7 Design Patterns Used

A pattern is used only when it removes a concrete problem. Each is listed with the problem it solves.

Pattern	Where	Problem it solves	Requirements
Composite	Document model: sections containing sections and blocks.	Reader and outline treat parts and wholes alike.	FR-26 to FR-31
Strategy	`DocumentExtractor`: PDF and PPTX extractors.	Choose the way to read a file at run time; add formats without editing the loader.	FR-01 to FR-07, NFR-M1
Adapter	Speech output, speech input and storage adapters.	Hide browser differences and failures behind our own interface.	NFR-C1 to C3, NFR-R1
Facade	`DocumentLoader`.	One simple call (`load`) hides validate, extract, clean and structure.	FR-01 to FR-07
Observer (publish–subscribe)	`EventBus`.	Lower layers inform upper layers without calling upward.	NFR-M2
Command	Voice and keyboard commands become `Command` objects with `execute()`.	The same action is reachable by key and by voice; unknown phrases are handled in one place.	FR-35 to FR-39
State	`PlaybackController` (Idle, Ready, Playing, Paused).	Operations behave correctly in each state; invalid actions are ignored or explained.	FR-18 to FR-25
Dependency injection	Application start-up.	Controllers receive interfaces, not concrete classes.	NFR-M3

7.1 Observer: Event Flow

Information goes down by calls and up by events. The next figure shows the main events. A publisher does not know who listens, which keeps the layers independent.

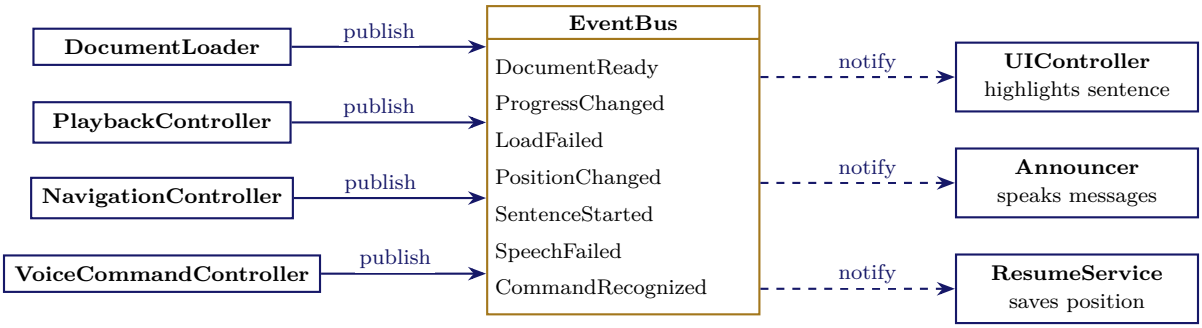


Figure 4: Observer pattern. Controllers publish events; the interface and the services subscribe.

8 Dynamic Model (Behaviour)

8.1 State of the Reader

Reading is the part of Hamnawa with the most state, so it is modelled as a state machine. `PlaybackController` holds the state and decides what each request means. Closing a document, from any state, returns the controller to *Idle*.

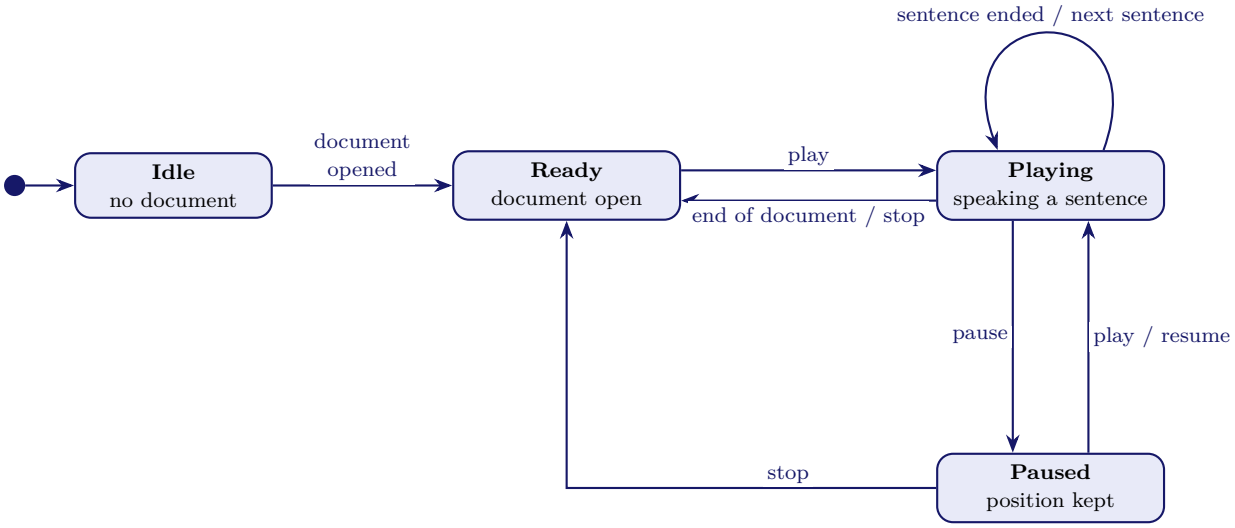


Figure 5: State diagram of `PlaybackController`. A speech failure moves Playing to Paused and announces the problem (FR-27, NFR-R1).

8.2 Interaction: Open a Document and Read It

The sequence below traces use cases UC-01 and UC-02: a user opens a file and starts reading. Read it from top to bottom.

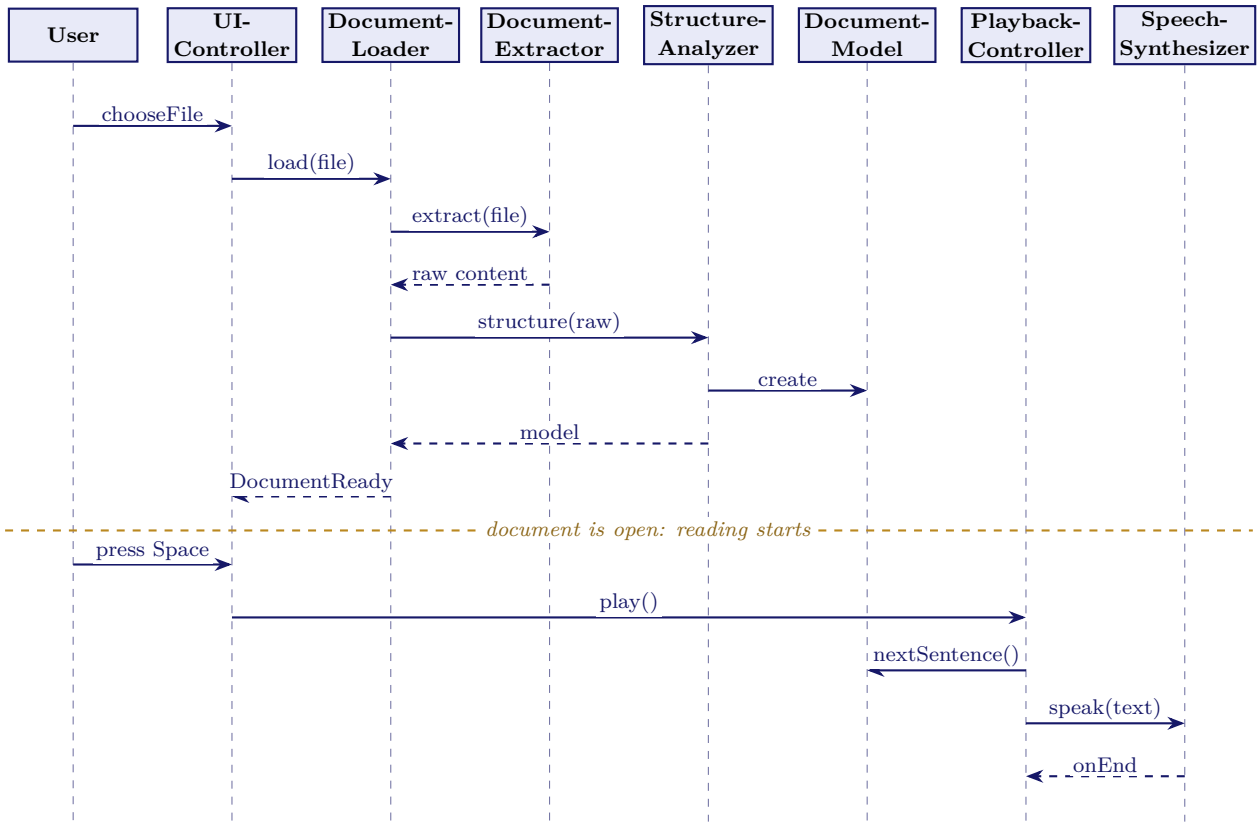


Figure 6: Sequence diagram: opening a document and reading aloud. Solid arrows are calls, dashed arrows are returns or events. Each call goes through an interface, so any participant can be a fake in tests.

Error path. If `extract(file)` fails (corrupt file, scanned PDF without text, file too large), the extractor reports a typed error, `DocumentLoader` publishes *LoadFailed*, and `Announcer` speaks a plain-language message and offers the next step. The user is never left in silence (NFR-R1, FR-47).

9 From Model to Code and Tests

9.1 Mapping to Source Files

Each class becomes one source file, named after the class, in the folder of its layer (see the Design Architecture Document). Interfaces are separate files, so a class can be read without reading its implementations.

Model element	Code element	Location
Class	TypeScript `class`	`src/<layer>/ClassName.ts`

Model element	Code element	Location
Interface	TypeScript `interface`	`src/domain/interfaces/` or the owning layer
Abstract class	`abstract class`	`src/domain/model/ContentBlock.ts`
Event	Typed message object	`src/shared/events.ts`
Test double	Class in the test folder	`tests/fakes/`

9.2 Testing Implications

- **Unit tests** for every class, using fakes for the four interfaces: for example, `PlaybackController` with `FakeSynthesizer`.
- **State tests** for each transition in Figure 5 (for example, *pause* when Idle has no effect and is explained).
- **Contract tests** run against both a real adapter and its fake, to show that the fake keeps the same promises (Liskov).
- **Scenario tests** follow the sequence in Figure 6 using the sample document set.

10 Trade-offs and Limitations

Trade-off	Effect	Mitigation
More classes and interfaces than a quick script	More files and some up-front design work.	Interfaces only at the four boundaries; no speculative classes.
Risk of over-using patterns	Needless complexity for a small team.	A pattern appears only with a stated problem (Section 7).
Events are harder to trace than calls	Debugging a chain of events is less direct.	Event log in development mode; a short, fixed list of event types.
Model may drift from code	Diagrams go out of date.	Same class names in code and diagrams; update this document at the end of every increment.
Classes may need to change when requirements are validated with users	Operations and states can change.	Design is incremental; review after each increment.

11 Traceability

Design decision	Supports	Requirements
Document model with Composite and polymorphic `cue()`	Format-independent reading; spoken structure	FR-23, FR-26 to FR-31
Extractor interface (Strategy)	Add formats; contain parse errors	FR-01 to FR-07, NFR-M1
Speech adapter interface	Browser differences hidden; failures reported	NFR-C1 to C3, NFR-R1
Command objects	Same action by key and voice	FR-35 to FR-39
State pattern in playback	Correct pause, resume, repeat, skip	FR-18 to FR-25
Observer event bus	Layers stay independent	NFR-M2
Storage interface	Local-only data; erase on request	NFR-S1, NFR-S2
Fakes for all interfaces	Tests without browser or microphone	NFR-M3

A Glossary

Class A template for objects: attributes (data) and operations (behaviour).

Interface A list of operations a class promises to offer, without code.

Abstract class A class that is not used directly but is specialised by subclasses.

Composition A strong "whole–part" link where the whole owns the part.

Polymorphism Calling the same operation on different types and getting type-specific behaviour.

Cohesion How strongly the parts of a class belong together.

Coupling How much a class depends on other classes.

Design pattern A named, reusable solution to a common design problem.

Dependency injection Giving a class its collaborators from outside rather than letting it create them.

Test double (fake) A simple stand-in for a real component, used in tests.

UML Unified Modeling Language, the standard notation for object-oriented models.

SOLID Five design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion.